SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

---

Code of Conduct:

I **VENNA MEENAKSHI REDDY (192471048)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **VENNA MEENAKSHI REDDY**

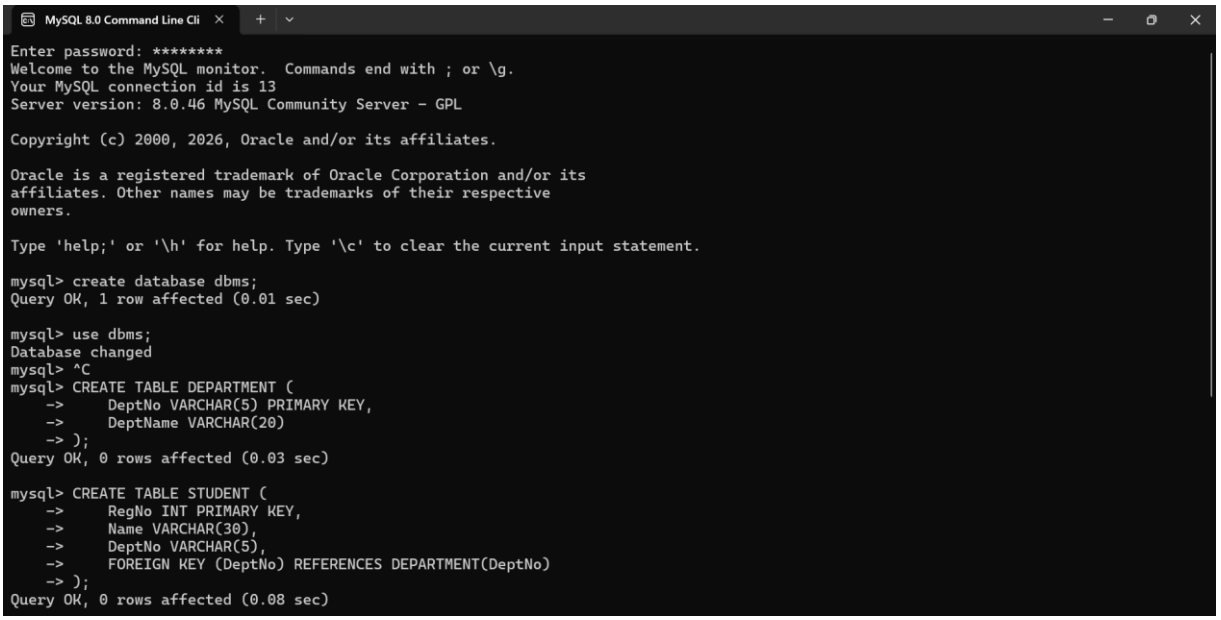
RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

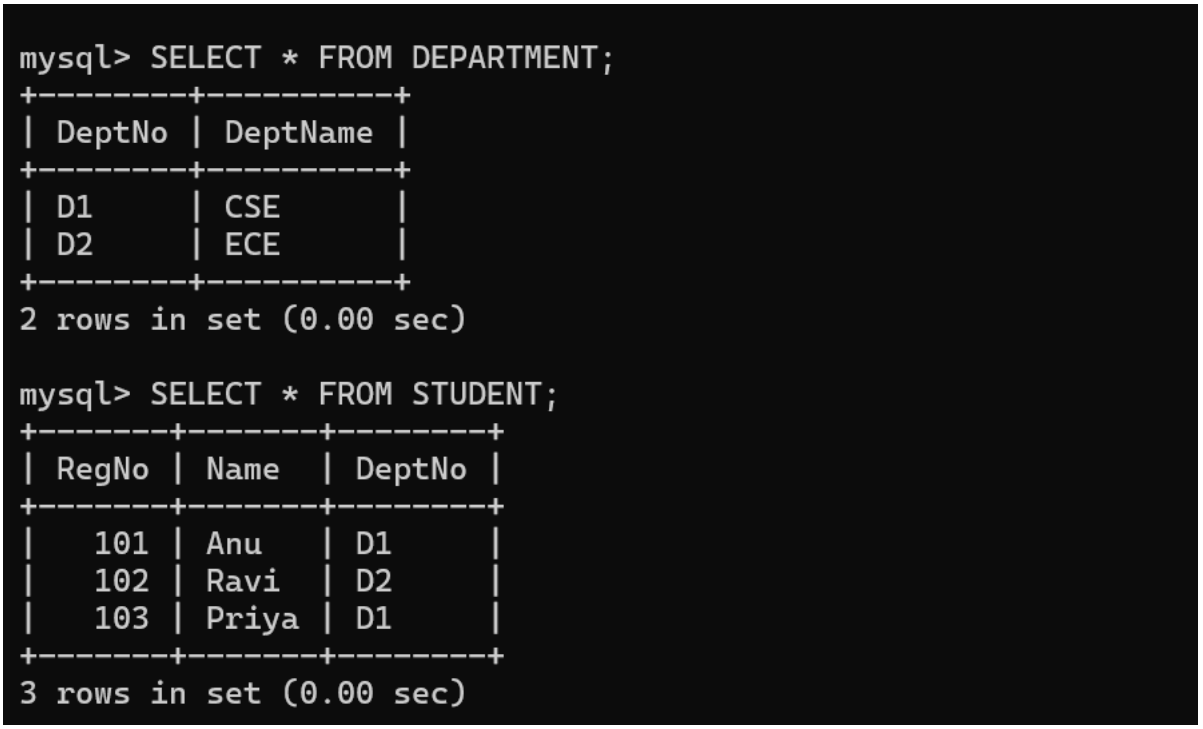
1.Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

OUTPUTS:



```
MySQL 8.0 Command Line Cli

Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database dbms;
Query OK, 1 row affected (0.01 sec)

mysql> use dbms;
Database changed
mysql> ^C
mysql> CREATE TABLE DEPARTMENT (
    ->     DeptNo VARCHAR(5) PRIMARY KEY,
    ->     DeptName VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE STUDENT (
    ->     RegNo INT PRIMARY KEY,
    ->     Name VARCHAR(30),
    ->     DeptNo VARCHAR(5),
    ->     FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO DEPARTMENT VALUES
    -> ('D1', 'CSE'),
    -> ('D2', 'ECE');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO STUDENT VALUES
    -> (101, 'Anu', 'D1'),
    -> (102, 'Ravi', 'D2'),
    -> (103, 'Priya', 'D1');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```



```
mysql> SELECT * FROM DEPARTMENT;
+--------+----------+
| DeptNo | DeptName |
+--------+----------+
| D1     | CSE      |
| D2     | ECE      |
+--------+----------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptNo |
+-------+-------+--------+
|   101 | Anu   | D1     |
|   102 | Ravi  | D2     |
|   103 | Priya | D1     |
+-------+-------+--------+
3 rows in set (0.00 sec)
```

(a) Using JOIN

```
mysql> SELECT S.RegNo, S.Name, D.DeptName
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
+-------+-------+----------+
| RegNo | Name  | DeptName |
+-------+-------+----------+
|   101 | Anu   | CSE      |
|   103 | Priya | CSE      |
|   102 | Ravi  | ECE      |
+-------+-------+----------+
3 rows in set (0.00 sec)

mysql>
```

Logic Used

- The STUDENT and DEPARTMENT tables are joined using the common attribute DeptNo.
- The INNER JOIN retrieves matching records from both tables.
- The query displays each student's Registration Number, Name, and Department Name.

(b) Using Subquery

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='CSE'
    -> );
+-------+
| Name  |
+-------+
| Anu   |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Logic

- The inner subquery retrieves the DeptNo for the CSE department.
- The outer query uses that DeptNo to search the STUDENT table.
- It displays the names of students belonging to the CSE department.

2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Notation	Operator Name	Explanation	Complex Query
σ (Sigma)	Selection	Selects rows that satisfy a given condition (similar to WHERE in SQL).	$\sigma_{DeptName='CSE' \wedge RegNo>101}(STUDENT)$
π (Pi)	Projection	Selects only the required columns/attributes (similar to SELECT in SQL).	$\pi_{Name,\ DeptName}(STUDENT \bowtie_{STUDENT.DeptNo=DEPARTMENT.DeptNo} DEPARTMENT)$
$\bowtie$ (Bowtie)	Join	Combines tuples from two relations based on a common attribute.	$STUDENT \bowtie_{STUDENT.DeptNo=DEPARTMENT.DeptNo} DEPARTMENT$
$\times$ (Cross)	Cartesian Product	Combines every tuple of one relation with every tuple of another relation.	$STUDENT \times COURSE$
$\cup$ (Union)	Union	Returns tuples from either of the two relations (duplicates removed).	$\pi_{RegNo}(CSE\_Students) \cup \pi_{RegNo}(ECE\_Students)$
$\cap$ (Intersection)	Intersection	Returns tuples common to both relations.	$CSE\_Students \cap ECE\_Students$
$-$ (Minus)	Set Difference	Returns tuples in first relation but not in second.	$All\_Students - Passed\_Students$
$\div$ (Division)	Division	Finds tuples in first relation related to all tuples in second relation.	$Enrollments \div Required\_Courses$
ρ (Rho)	Rename	Renames a relation or its attributes.	$\rho_{Emp}(EmpID, EName, DeptNo)(EMPLOYEE)$
$\leftarrow$ (Assignment)	Assignment	Stores the result of an expression into a temporary relation.	$R_1 \leftarrow \sigma_{DeptName='CSE'}(DEPARTMENT)$
$\wedge$ (AND)	Logical AND	Both conditions must be true.	$\sigma_{DeptName='CSE' \wedge RegNo>101}(STUDENT)$
$\vee$ (OR)	Logical OR	At least one condition must be true.	$\sigma_{DeptName='CSE' \vee DeptName='ECE'}(DEPARTMENT)$
$\neg$ (NOT)	Logical NOT	Negates a condition.	$\sigma_{\neg(DeptName='CSE')}(DEPARTMENT)$
$=$ (Equal To)	Equal To	Checks whether two values are equal.	$\sigma_{DeptNo='D1'}(STUDENT)$
$\neq$ (Not Equal To)	Not Equal To	Checks whether two values are not equal.	$\sigma_{DeptNo\neq'D1'}(STUDENT)$
$>, <, \geq, \leq$	Comparison Operators	Compare numeric or character values.	$\sigma_{RegNo>100 \wedge RegNo\leq200}(STUDENT)$

3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

Scenario

A college wants to maintain student and department information. The system should store student details, department details, retrieve student information, and update records when necessary.

Relational Schema

DEPARTMENT

DEPARTMENT(

DeptNo PRIMARY KEY,

DeptName

)

STUDENT

STUDENT(

RegNo PRIMARY KEY,

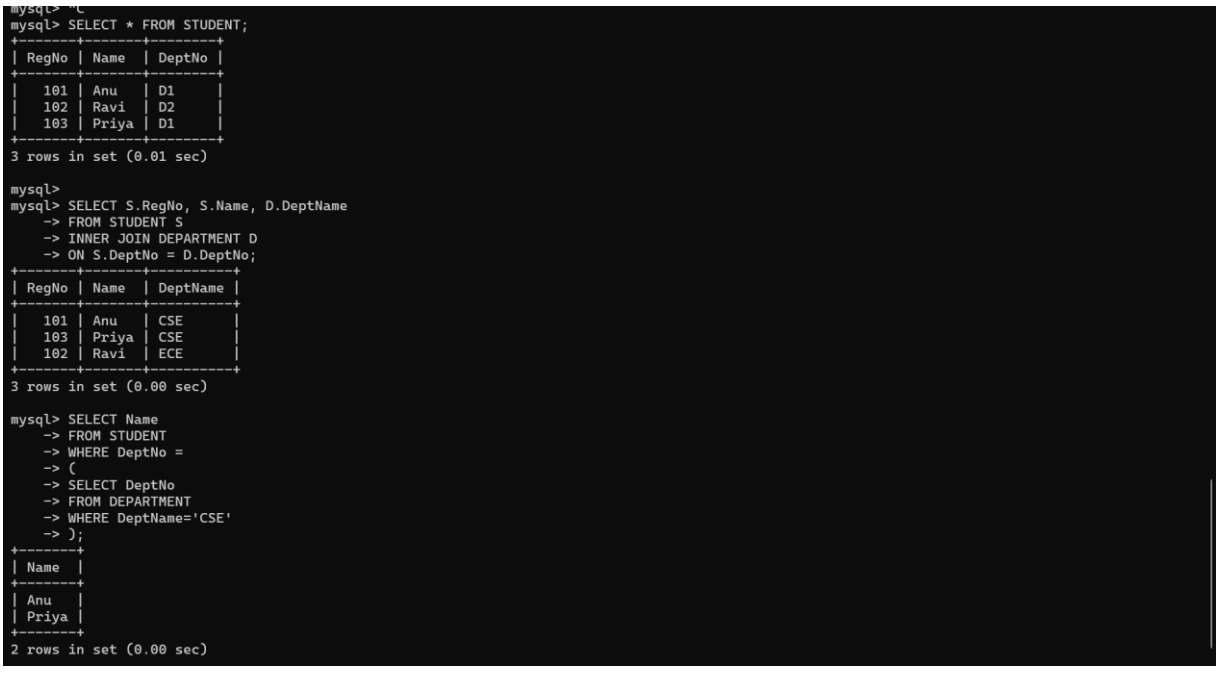
Name,

DeptNo FOREIGN KEY REFERENCES DEPARTMENT(DeptNo)

)

OUTPUTS:

1) Data Retrieval Queries



```
mysql> ^C
mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptNo |
+-------+-------+--------+
|   101 | Anu   | D1     |
|   102 | Ravi  | D2     |
|   103 | Priya | D1     |
+-------+-------+--------+
3 rows in set (0.01 sec)

mysql>
mysql> SELECT S.RegNo, S.Name, D.DeptName
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
+-------+-------+----------+
| RegNo | Name  | DeptName |
+-------+-------+----------+
|   101 | Anu   | CSE      |
|   103 | Priya | CSE      |
|   102 | Ravi  | ECE      |
+-------+-------+----------+
3 rows in set (0.00 sec)

mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='CSE'
    -> );
+-------+
| Name  |
+-------+
| Anu   |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

2) Data Update Queries

Update student's department:

```
mysql> UPDATE STUDENT
    -> SET DeptNo='D2'
    -> WHERE RegNo=103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptNo |
+-------+-------+--------+
|   101 | Anu   | D1     |
|   102 | Ravi  | D2     |
|   103 | Priya | D2     |
+-------+-------+--------+
3 rows in set (0.00 sec)

mysql>
```

3) Insert a new student

```
mysql> INSERT INTO STUDENT
    -> VALUES (104,'Kiran','D1');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptNo |
+-------+-------+--------+
|   101 | Anu   | D1     |
|   102 | Ravi  | D2     |
|   103 | Priya | D2     |
|   104 | Kiran | D1     |
+-------+-------+--------+
4 rows in set (0.00 sec)
```

4) Delete a student

```
mysql> DELETE FROM STUDENT
    -> WHERE RegNo=104;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptNo |
+-------+-------+--------+
|   101 | Anu   | D1     |
|   102 | Ravi  | D2     |
|   103 | Priya | D2     |
+-------+-------+--------+
3 rows in set (0.00 sec)
```

Explanation

- The DEPARTMENT table stores department information.
- The STUDENT table stores student details and references the department using a foreign key.
- SELECT queries retrieve student and department information.
- UPDATE modifies existing records.
- INSERT adds new records.
- DELETE removes records.

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

Explanation of each step of execution:

Step 1: Execute the Innermost Subquery

```
SELECT DeptName
FROM DEPARTMENT
WHERE DeptNo='D1';
```

Output

DeptName
CSE

Explanation:

- SQL first executes the innermost subquery.
- It searches the DEPARTMENT table for DeptNo = 'D1'.
- The matching department name is CSE.
- This value (CSE) is passed to the next subquery.

Step 2: Execute the Second Subquery

The second subquery becomes:

```
SELECT DeptNo
FROM DEPARTMENT
WHERE DeptName='CSE';
```

Output

DeptNo
D1

Explanation:

- SQL now searches the DEPARTMENT table for DeptName = 'CSE'.
- It finds the corresponding DeptNo, which is D1.
- This value (D1) is passed to the outer query.

Step 3: Execute the Outer Query

The outer query becomes:

SELECT Name

FROM STUDENT

WHERE DeptNo='D1';

Output

Name
Anu
Priya

Explanation:

- SQL searches the **STUDENT** table for all students whose **DeptNo = 'D1'**.
- Two students satisfy this condition:
    - Anu
    - Priya
- Their names are displayed as the final result.

Final Output

Name
Anu
Priya

FINAL OUTPUT:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='CSE'
    -> );
+-------+
| Name  |
+-------+
| Anu   |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

OUTPUT:

Using JOIN

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> WHERE D.DeptName = 'CSE';
+------+
| Name |
+------+
| Anu  |
+------+
1 row in set (0.00 sec)

mysql>
```

Using Subquery

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    ->     SELECT DeptNo
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+------+
| Name |
+------+
| Anu  |
+------+
1 row in set (0.00 sec)
```

Comparison

Feature	JOIN	Subquery
Tables Used	STUDENT, DEPARTMENT	STUDENT, DEPARTMENT
Performance	Faster for large datasets	May be slower due to nested execution
Readability	Better for combining multiple tables	Simpler for single-value lookups
Execution	Retrieves data in a single query using JOIN	Executes the inner query first, then the outer query
Best Used For	Retrieving related data from multiple tables	Filtering data based on another query

Efficiency Evaluation

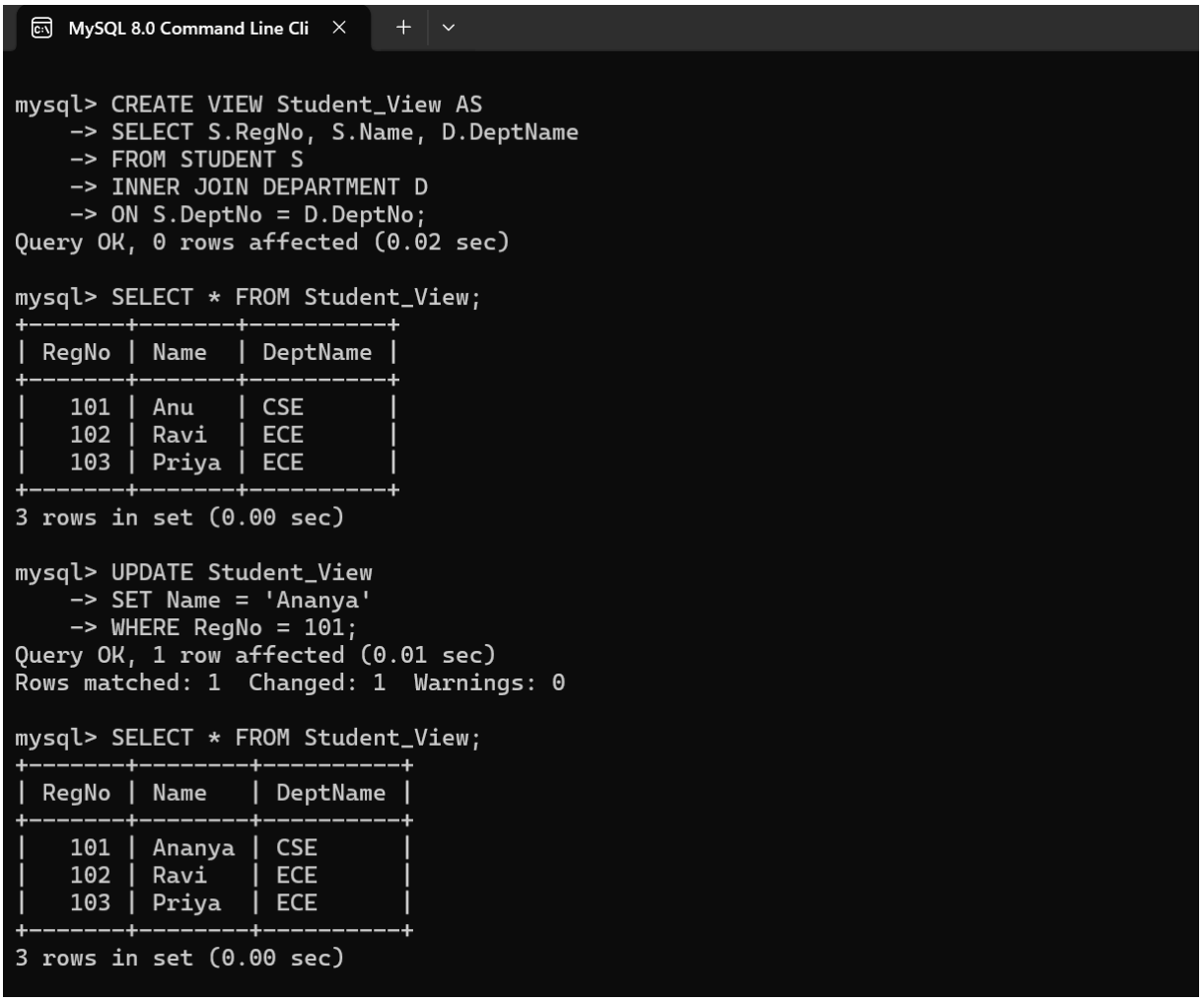
- JOIN is generally more efficient because it combines related tables in a single query execution.
- Subqueries are useful for filtering data but may require additional processing, especially when nested.
- For large databases, JOIN usually provides better performance and is easier to optimize.
- For simple filtering conditions, a subquery is easy to understand and write.

Conclusion

Both JOIN and Subquery produce the same output for this problem. However, JOIN is generally the preferred approach because it is faster, more efficient for large datasets, and easier to use when retrieving data from multiple related tables. Therefore, JOIN is recommended for most real-world database applications, while Subqueries are suitable for simple filtering operations.

6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

OUTPUT:



Justification

Security

- A view restricts access to selected columns instead of exposing the entire table.
- Users can access only the required information.
- Sensitive data remains protected.

Abstraction

- A view hides the complexity of SQL queries and joins.
- Users can retrieve data without knowing the underlying table structure.
- It provides a simplified interface for accessing the database.

Advantages of Views

- Restricts unauthorized access to data.
- Improves database security.
- Simplifies complex queries.
- Provides data abstraction.
- Reuses frequently used queries.
- Makes database maintenance easier.

Conclusion

Views provide restricted data access by displaying only the required information while hiding the underlying tables. They improve security by limiting access to sensitive data and support abstraction by simplifying complex SQL queries, making the database easier and safer to use.

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Relational Algebra Expression

$$\pi_{Name}\left(\sigma_{DeptName='CSE' \wedge RegNo>101}\left(STUDENT \bowtie_{STUDENT.DeptNo=DEPARTMENT.DeptNo} DEPARTMENT\right)\right)$$

SQL query:

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> WHERE D.DeptName = 'CSE';
+--------+
| Name   |
+--------+
| Ananya |
+--------+
1 row in set (0.00 sec)
```

Analysis of Representation

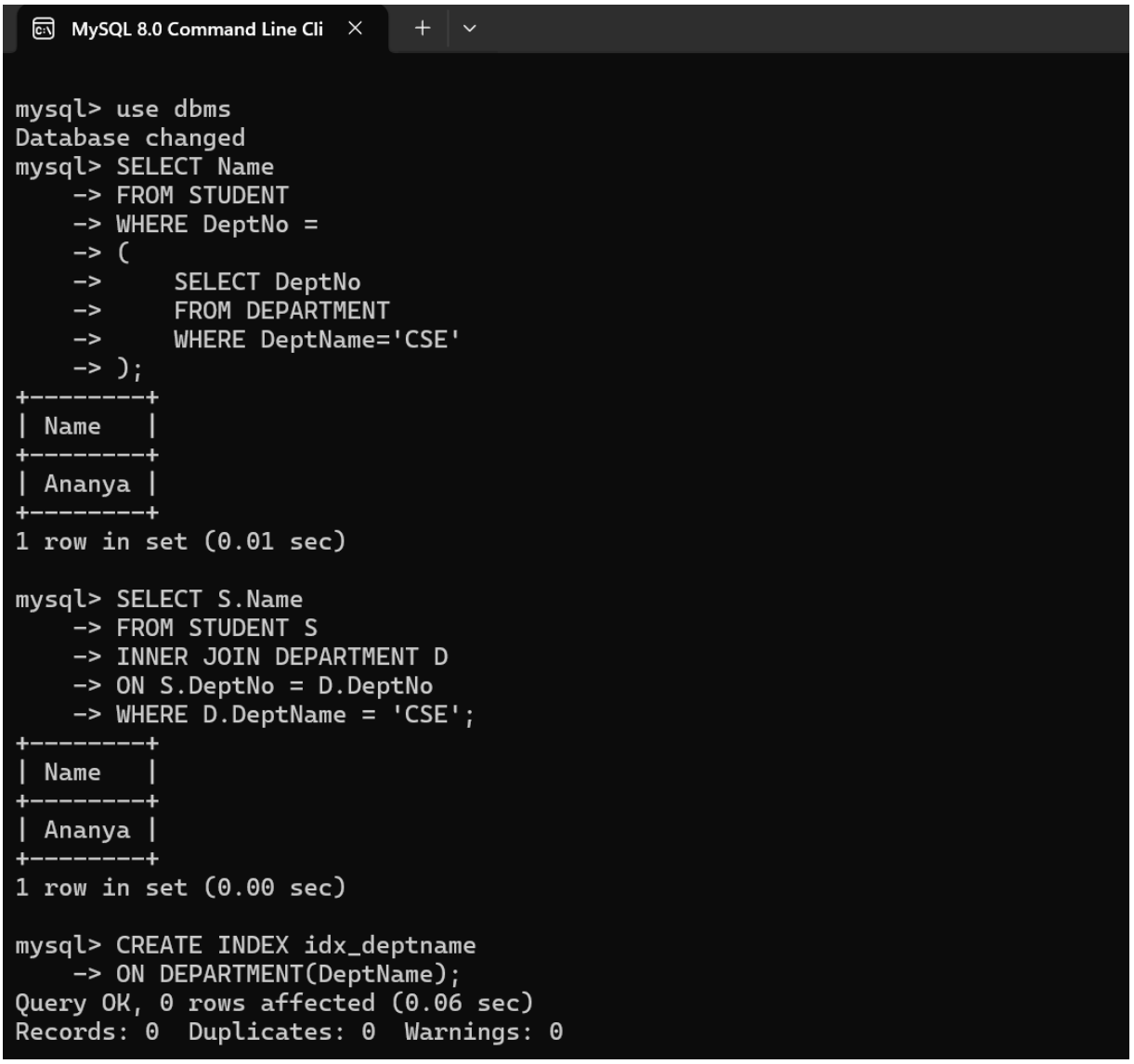
Relational Algebra	SQL
Uses mathematical operators such as π (Projection), σ (Selection), and ⋈ (Join).	Uses SQL keywords such as SELECT, FROM, WHERE, and JOIN.
Describes the logical sequence of operations.	Provides executable commands to retrieve data from the database.
Used for query analysis and optimization.	Used for implementing and executing queries in a DBMS.
Not directly executable.	Directly executable in MySQL and other DBMSs.

Conclusion

The relational algebra expression represents the query using mathematical operators, while the SQL query implements the same logic using SQL syntax. Both produce the same result, but SQL is executable in a DBMS, whereas relational algebra is primarily used for query design and analysis. This example demonstrates how a relational algebra expression can be translated into an equivalent SQL query while preserving the same query logic.

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

OUTPUT:



```
mysql> CREATE INDEX idx_deptno
    -> ON STUDENT(DeptNo);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE VIEW Student_View AS
    -> SELECT S.RegNo,
    ->        S.Name,
    ->        D.DeptName
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
ERROR 1050 (42S01): Table 'Student_View' already exists
mysql> ^C
mysql> SELECT Name
    -> FROM Student_View
    -> WHERE DeptName='CSE';
+--------+
| Name   |
+--------+
| Ananya |
+--------+
1 row in set (0.00 sec)

mysql>
```

1. Inefficient Query (Using Subquery)

Drawback

- Executes the subquery before the main query.
- May be slower for large datasets.

2. Optimized Query (Using JOIN)

Improvement

- Retrieves data using a single JOIN operation.
- Better performance for large databases.
- Easier to optimize by the DBMS.

3. Optimization Using Index

Improvement

- Faster searching using indexed columns.
- Reduces table scanning.
- Improves JOIN performance.

4. Optimization Using View

Improvement

- Simplifies complex queries.
- Improves code reusability.
- Provides data abstraction.

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

OUTPUT:

```
mysql> SELECT D.DeptName,
    ->        COUNT(S.RegNo) AS Total_Students
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> GROUP BY D.DeptName
    -> HAVING COUNT(S.RegNo) =
    -> (
    ->     SELECT MAX(Student_Count)
    ->     FROM
    ->     (
    ->         SELECT COUNT(RegNo) AS Student_Count
    ->         FROM STUDENT
    ->         GROUP BY DeptNo
    ->     ) AS Dept_Count
    -> );
+----------+----------------+
| DeptName | Total_Students |
+----------+----------------+
| ECE      |              2 |
+----------+----------------+
1 row in set (0.01 sec)

mysql>
```

Explanation

Step 1

Join the STUDENT and DEPARTMENT tables using DeptNo.

Step 2

Group the records by DeptName using GROUP BY.

Step 3

Count the number of students in each department using COUNT().

Step 4

The nested subquery finds the maximum student count among all departments.

Step 5

The HAVING clause compares each department's student count with the maximum value and displays only the department having the highest number of students.

Conclusion

This query solves a real-world problem by identifying the department with the highest student enrollment. It combines JOIN, GROUP BY, aggregate functions (COUNT, MAX), the HAVING clause, and a nested subquery to efficiently retrieve the required result.

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

OUTPUT:

Approach 1: Using JOIN

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> INNER JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> WHERE D.DeptName = 'CSE';
+--------+
| Name   |
+--------+
| Ananya |
+--------+
1 row in set (0.00 sec)

mysql>
```

Reason for Choosing JOIN:

Reason

- Combines data from related tables.
- Efficient for retrieving information from multiple relations.
- Best choice for this query.

Approach 2: Using Subquery

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    ->     SELECT DeptNo
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+--------+
| Name   |
+--------+
| Ananya |
+--------+
1 row in set (0.00 sec)
```

Reason for Choosing Subquery:

- Retrieves data using a nested query.
- Suitable for simple filtering conditions.
- May be less efficient than a JOIN for large datasets.

Approach 3: Using View

```
mysql> SELECT Name
    -> FROM Student_View
    -> WHERE DeptName = 'CSE';
+--------+
| Name   |
+--------+
| Ananya |
+--------+
1 row in set (0.00 sec)
```

Reason for Choosing View:

- Simplifies frequently used queries.
- Provides restricted access to selected data.
- Improves readability and reusability.

Analysis

Approach	Suitable When	Advantages	Limitations
JOIN	Retrieving related data from multiple tables	Fast, efficient, easy to optimize	Slightly more complex syntax
Subquery	Filtering data based on another query	Easy to understand for simple conditions	Can be slower for large datasets
View	Repeatedly accessing the same query results	Improves security, abstraction, and reusability	Requires creating and maintaining the view

Justification

For this requirement, JOIN is the most suitable choice because it efficiently combines the STUDENT and DEPARTMENT tables and retrieves the required data in a single query. Subqueries are useful for simple filtering operations, while views are preferred when the same query is executed repeatedly or when restricted access and abstraction are required.

Conclusion

The choice between JOIN, Subquery, and View depends on the query requirements. For retrieving related data from multiple tables, JOIN is the preferred approach due to its efficiency. Subqueries are appropriate for filtering operations, whereas views are best suited for simplifying repeated queries and providing secure, abstract access to database information.